Cuida Tu Agua: plataforma IoT web y móvil para el monitoreo del consumo de agua y el control remoto de la válvula basada en microservicios

Juan Esteban Ome Esquivel
Aprendiz
SENA
Regional Huila, Colombia
juanome43@gmail.com

Juan Manuel Gómez
Aprendiz
SENA
Regional Huila, Colombia
juanmanuel115774@gmail.com

Diego Alejandro Alvarado
Aprendiz
SENA
Regional Huila, Colombia
daalvarado2007@gmail.com

Carlos Julio Cadena
Instructor
SENA
Regional Huila, Colombia
cjcadena@sena.edu.co

RESUMEN

Este trabajo describe el desarrollo de Cuida Tu Agua, una plataforma web y móvil que mide en tiempo real el consumo de agua de hogares y negocios mediante un medidor IoT instalado en el registro de entrada, alerta cuando el medidor se desconecta, y permite cerrar la válvula de forma remota con una confirmación de seguridad. El medidor usa un ESP32, un sensor de caudal YF-S201 y una electroválvula normalmente cerrada, y se comunica por MQTT. El backend está formado por seis microservicios con arquitectura hexagonal (Java 21 con Spring Boot y .NET 10), integrados por eventos en RabbitMQ, y los clientes se construyeron con React Native y Expo para Android, iOS y web. El proyecto se gestionó con Scrum a partir de un backlog de 69 historias de usuario. La entrega se centró en medir el agua por lugar: al corte del 15 de octubre de 2026 estaban implementadas 40 historias, entre ellas todas las de prioridad Must del alcance actual, y las otras 29 quedaron como mejoras de una versión 2; las 626 pruebas automatizadas ejecutadas (JUnit, xUnit, Jest y un arnés del firmware) pasaron sin fallas, con coberturas superiores al 90 % en la mayoría de las capas de dominio y aplicación. Los resultados muestran que separar el dominio de la tecnología facilita probar reglas críticas, como el bloqueo por intentos fallidos o la confirmación de órdenes de la válvula, sin depender del hardware.

Artículo elaborado como producto del proyecto formativo Cuida Tu Agua, programa Tecnología en Análisis y Desarrollo de Software, Servicio Nacional de Aprendizaje (SENA).
SENA, octubre de 2026, Colombia
© 2026 Los autores.

CONCEPTOS CCS

Computer systems organization → Embedded and cyber-physical systems • Software and its engineering → Software architectures • Software and its engineering → Software testing and debugging

PALABRAS CLAVE

Internet de las cosas, medición inteligente de agua, microservicios, arquitectura hexagonal, MQTT, ESP32, detección de fugas, Scrum

Formato de referencia ACM:

Juan Esteban Ome Esquivel, Juan Manuel Gómez, Diego Alejandro Alvarado y Carlos Julio Cadena. 2026. Cuida Tu Agua: plataforma IoT web y móvil para el monitoreo del consumo de agua y el control remoto de la válvula basada en microservicios. Servicio Nacional de Aprendizaje (SENA), Colombia.

1 INTRODUCCIÓN

1.1 El problema

En la mayoría de hogares y negocios colombianos el consumo de agua solo se conoce una vez al mes, cuando llega el recibo del acueducto. Entre un recibo y otro no hay forma práctica de saber cuánta agua se usa, en qué momento del día ni si existe una fuga interna, como un sanitario que no cierra o una tubería rota dentro de un muro. Ese tipo de pérdida puede pasar semanas sin detectarse y solo se descubre cuando el valor del recibo aumenta.

Las pérdidas también son un problema del sector. La Superintendencia de Servicios Públicos Domiciliarios reportó para la vigencia 2024 un índice nacional de pérdidas por usuario facturado de 10,34 m³ por suscriptor al mes, por encima del estándar regulatorio de 6 m³, calculado con la información de 294 áreas de prestación [1]. Aunque ese indicador corresponde a las redes de las empresas y no a las instalaciones internas, muestra que el agua que se pierde sin control es un problema relevante en el país.

A esto se suma que, cuando el dueño de un lugar no está presente, no tiene cómo cortar el agua ante un daño, ni herramientas para comprobar si lo que le factura el acueducto coincide con lo que realmente consumió.

1.2 Motivación

La información oportuna sobre el consumo cambia el comportamiento de las personas. En un estudio con hogares de Sacramento, quienes recibieron reportes que comparaban su consumo, medido con medidores inteligentes, con el de hogares similares redujeron su uso de agua cerca de un 8 % durante los seis meses siguientes [2]. Si un reporte periódico produce ese efecto, ver el consumo en tiempo real y recibir alertas inmediatas puede ayudar todavía más a reaccionar ante fugas y consumos anormales.

Desde el punto de vista formativo, el proyecto permitió al equipo aplicar en un caso real las competencias del programa de Análisis y Desarrollo de Software del SENA: levantamiento de requisitos, modelado, arquitectura de software, desarrollo backend, móvil y web, integración con hardware, pruebas y documentación.

1.3 Objetivos

El objetivo general fue desarrollar una plataforma web y móvil, integrada con un medidor IoT, que permita a hogares y negocios monitorear su consumo de agua en tiempo real, recibir alertas y controlar de forma remota la válvula de entrada. Los objetivos específicos fueron: construir un medidor de bajo costo con ESP32, sensor de caudal y electroválvula normalmente cerrada; implementar el backend como microservicios con arquitectura hexagonal; desarrollar las aplicaciones móvil y web y el panel de administración con un mismo sistema de diseño; notificar al usuario por varios canales según la urgencia; y validar el sistema con pruebas automatizadas en todos sus componentes.

1.4 Contribuciones

- Un medidor IoT con ESP32 y válvula normalmente cerrada que conserva su estado ante cortes de energía y guarda hasta una hora de lecturas cuando pierde la red.
- Una arquitectura de seis microservicios con arquitectura hexagonal, integrados por eventos, que separa el dominio de la tecnología.
- Un mecanismo de cierre remoto de la válvula con código de confirmación por correo y confirmación del medidor en 30 segundos.
- Una estrategia de pruebas con 626 pruebas automatizadas que incluye la ejecución del firmware real contra un broker MQTT.

2 TRABAJOS RELACIONADOS

2.1 Monitoreo de redes de agua con IoT

Velayudhan et al. [3] revisan cómo las tecnologías IoT apoyan el monitoreo y el control en las distintas etapas de la distribución de agua, y comparan sensores, protocolos de comunicación y plataformas. Su revisión muestra que la mayoría de soluciones se enfocan en la red de la empresa prestadora; Cuida Tu Agua se ubica en el último tramo, dentro de la instalación del usuario, donde el interesado directo es el dueño del lugar.

2.2 Detección de fugas

Farah y Shahrour [4] comparan los métodos de detección de fugas, desde los acústicos y de balance de masas hasta los basados en datos, según su sensibilidad, costo y tasa de falsas alarmas. Chang et al. [5] proponen un sistema con ESP32 y dos medidores de efecto Hall que detecta fugas comparando el caudal entre ellos, envía alertas por mensajería y cifra los mensajes MQTT. Cuida Tu Agua usa un solo sensor a la entrada, por lo que su detección de fugas, planeada para la versión 2, se basará en la duración del caudal continuo; en la versión actual aporta el cierre remoto de la válvula y la gestión de varios lugares por usuario.

2.3 Protocolos de comunicación para IoT

Naik [6] compara MQTT, CoAP, AMQP y HTTP para sistemas IoT y destaca que MQTT tiene un encabezado pequeño, un modelo de publicación y suscripción y niveles de calidad de servicio adecuados para dispositivos con recursos limitados. El proyecto usa MQTT 3.1.1 [7] entre el medidor y el servidor, porque permite que el servidor envíe órdenes al medidor sin que este consulte, y AMQP mediante RabbitMQ para los eventos entre servicios.

2.4 Microservicios y arquitectura hexagonal

Newman [8] describe los microservicios como servicios pequeños, desplegables de forma independiente y organizados alrededor de capacidades del negocio. La arquitectura hexagonal, o de puertos y adaptadores, propuesta por Cockburn [9], aísla la lógica de la aplicación de los mecanismos externos (interfaz, base de datos, mensajería), lo que permite probarla sin ellos. El proyecto combina ambos enfoques: microservicios en el nivel global y arquitectura hexagonal dentro de cada servicio.

3 METODOLOGÍA

3.1 Gestión del proyecto

El proyecto se gestionó con Scrum [10] en sprints semanales, con planeación, reunión diaria, refinamiento, revisión y retrospectiva. El Product Backlog v2.1 tiene 69 historias de usuario agrupadas en 15 épicas, priorizadas con el método MoSCoW (24 Must, 34 Should, 9 Could y 2

Won't) y estimadas en 279 puntos de historia, repartidas en cuatro entregas. El producto mínimo viable se limitó a cuatro épicas: cuentas, lugares, vinculación del medidor y monitoreo del consumo.

3.2 Requisitos y modelado

Los requisitos se documentaron en una especificación según la práctica recomendada IEEE 830 [11], con un requisito funcional por historia de usuario, 16 requisitos no funcionales y 14 reglas de negocio. El comportamiento se modeló con 39 diagramas UML (casos de uso, secuencia, actividad, estados, clases, componentes y despliegue) y siete diagramas entidad-relación del modelo físico. Las pantallas se prototiparon en alta fidelidad (58 pantallas para móvil, web y administración) antes de implementarlas.

3.3 Estrategia de pruebas

Las pruebas se organizaron por niveles: pruebas unitarias del dominio y de los casos de uso con dobles de prueba escritos a mano; pruebas de integración de infraestructura con SQLite en memoria; pruebas del contrato HTTP; pruebas de la aplicación con Jest; y un arnés que ejecuta el firmware real en un computador contra un broker Mosquitto con la configuración del proyecto. La calidad se evaluó con indicadores del modelo ISO/IEC 25010 [12], en especial la adecuación funcional (historias implementadas), la fiabilidad (pruebas aprobadas) y la mantenibilidad (cobertura y separación de capas).

4 DISEÑO DEL SISTEMA

4.1 Arquitectura general

El sistema se organiza en capas (figura 1). Los clientes consumen una API REST protegida con tokens JWT; el medidor se comunica solo con el servicio de dispositivos a través del broker MQTT; los microservicios guardan sus datos en SQL Server, cada uno en su propio schema, consultan Redis para rechazar tokens revocados y se integran por eventos en RabbitMQ. La Tabla 1 resume la responsabilidad de cada microservicio.

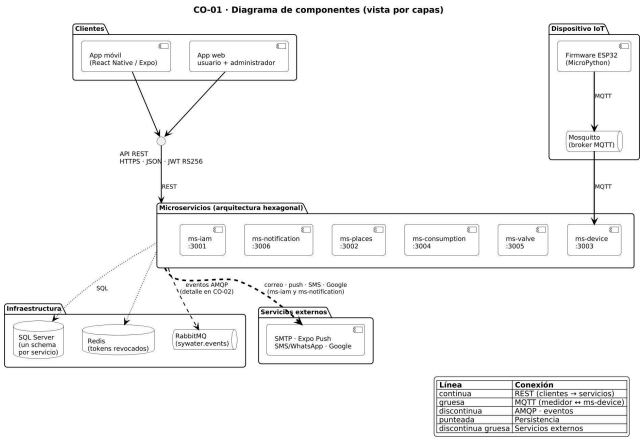


Figura 1: Arquitectura de Cuida Tu Agua por capas

Tabla 1: Microservicios del sistema

Servicio	Tecnología	Responsabilidad
ms-iam	Java 21, Spring Boot	Cuentas, sesiones, perfil, códigos de acción y administración de usuarios
ms-places	.NET 10	Lugares, catálogo geográfico, tarifas y consejos de ahorro
ms-device	.NET 10, MQTTnet	Medidores: vinculación, estado, telemetría y órdenes
ms-consumption	.NET 10	Lecturas, consumo por período y caudal actual
ms-valve	.NET 10	Estado de la válvula, órdenes e historial
ms-notification	.NET 10	Notificaciones, preferencias, correo y push

4.2 Medidor IoT

El medidor combina un ESP32, un sensor de caudal YF-S201 de efecto Hall y una electroválvula normalmente cerrada accionada por un relé. Esta válvula necesita energía para dejar pasar el agua y se cierra sola ante un corte de luz; el firmware guarda el último estado ordenado y lo restaura al reiniciar, de modo que una válvula cerrada sigue cerrada. El firmware, escrito en MicroPython, envía un latido cada 30 segundos y una lectura cada 10 segundos con el caudal, el volumen del intervalo y el total acumulado; si pierde la red, guarda hasta una hora de lecturas y las envía al reconectarse.

La primera configuración se hace sin cables: el medidor crea una red WiFi propia y una página web en la que el usuario elige la red de su casa. Cada medidor tiene un serial, un token secreto y un código de emparejamiento de

8 caracteres; el servidor solo guarda sus hash, y la etiqueta de la caja lleva el serial, el código y un QR, nunca el token.

4.3 Integración entre servicios

Los servicios se integran de tres formas (figura 2). El servicio de dispositivos publica cuatro eventos (medidor vinculado, desvinculado, lectura recibida y estado de la válvula reportado) en un exchange de RabbitMQ, del que los consumen los servicios de consumo, válvula y notificaciones. Las llamadas HTTP internas, protegidas con una clave compartida, se usan cuando un servicio necesita una respuesta inmediata, como confirmar que un lugar pertenece al usuario. Los consumidores son idempotentes y los mensajes que fallan cinco veces van a una cola de mensajes muertos.

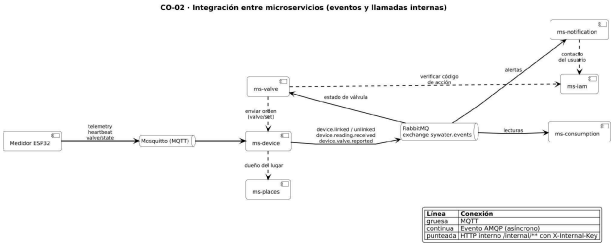


Figura 2: Integración por eventos y llamadas internas entre microservicios

4.4 Seguridad

El servicio de cuentas emite tokens JWT [13] firmados con RS256, de una hora, y tokens de renovación de siete días que rotan en cada uso; los demás servicios los validan solo con la llave pública. Al cerrar sesión, bloquear o eliminar una cuenta, el token se registra en una lista de revocación en Redis. Las contraseñas se guardan con BCrypt, el inicio de sesión se bloquea 15 minutos después de cinco intentos fallidos y la vinculación del medidor se bloquea después de cinco códigos incorrectos. Como cerrar la válvula puede dejar a una familia sin agua, la orden exige un código de seis dígitos enviado al correo, válido cinco minutos y de un solo uso. Los errores se responden en formato Problem Details [14] con un código estable que la aplicación traduce al español.

4.5 Patrones de diseño

Cada microservicio separa cuatro capas: dominio, aplicación, infraestructura y API. Además de los puertos y adaptadores, el código aplica raíces de agregado y objetos de valor, métodos de fábrica, objetos de política para los umbrales configurables, repositorios con mapeadores de datos, publicador y suscriptor con mensajes muertos, outbox transaccional para el correo con reintentos crecientes, consumidores idempotentes basados en índices únicos y trabajadores en segundo plano para marcar medidores desconectados y vencer órdenes sin respuesta.

5 IMPLEMENTACIÓN

5.1 Backend

El servicio de cuentas se implementó en Java 21 con Spring Boot 4.1.1, Spring Security como servidor de recursos OAuth2 y Spring Data JPA. Los otros cinco servicios se implementaron en .NET 10 con ASP.NET Core y Entity Framework Core, con MQTTnet para el broker y RabbitMQ.Client para los eventos. En total se expusieron 65 endpoints REST. La base de datos tiene 31 tablas en seis schemas, cuyas migraciones se versionan con Liquibase en repositorios separados, con etiqueta y rollback por versión.

5.2 Aplicación móvil y web

Los clientes se construyeron con React Native, Expo y TypeScript. La misma aplicación funciona en Android, iOS y el navegador: los archivos con sufijo .web solo se cargan en la web, lo que permitió adaptar la interfaz al escritorio (menú lateral, cuadrículas y diálogos centrados) sin modificar la aplicación móvil. El panel de administración vive en la versión web y solo aparece para cuentas con rol de administrador. La sesión se renueva sola mediante interceptores que evitan renovaciones simultáneas (figura 3).



Figura 3: Panel del lugar en la aplicación móvil: consumo del día, caudal actual y estado de la válvula

5.3 Detección de eventos

Un trabajador de ms-device marca como desconectado un medidor que no reporta en 10 minutos, y ms-notification envía una sola alerta por desconexión, que se rearma con la siguiente lectura. Las órdenes de la válvula se confirman cuando el medidor reporta el estado esperado con el mismo identificador de orden; si no lo hace en 30 segundos, la orden se marca como sin respuesta y el estado de la válvula no cambia. Las alertas se clasifican en críticas, importantes e informativas, y el usuario elige por qué canales recibir cada nivel, aunque las críticas siempre llegan a la aplicación.

6 RESULTADOS

6.1 Avance funcional

Al 15 de octubre de 2026 estaban implementadas 40 de las 69 historias de usuario (161 de 279 puntos de historia). La entrega se centró en el núcleo del producto, la medición de agua por lugar: cuentas, lugares, medidor, consumo, válvula, notificaciones, tarifas y administración. Las otras 29 historias (118 puntos) quedaron especificadas como mejoras de una versión 2: la alerta de fuga y el cierre automático, las metas de consumo, el acceso de invitados y colaboradores, los recibos y los reportes. Entre ellas hay cuatro historias Must (HU-004, HU-029, HU-041 y HU-045) que el equipo reprogramó en la revisión del sprint, de modo que todas las historias Must del alcance actual están implementadas.

Después de ese corte, el equipo completó las metas de consumo por lugar (HU-023 y HU-024), el acceso de invitados y colaboradores (HU-039 a HU-047) y las alertas críticas por WhatsApp y SMS (HU-028), y los servicios pasaron a ejecutarse en contenedores Docker. Esas funciones no están incluidas en las cifras ni en las pruebas de este artículo, que corresponden al corte del 15 de octubre y a la ejecución del 10 de octubre.

6.2 Pruebas

Las pruebas automatizadas se ejecutaron el 10 de octubre de 2026 sobre una copia del repositorio. La Tabla 2 resume los indicadores obtenidos. Las 626 pruebas ejecutadas pasaron sin fallas; quedaron sin ejecutar 42 pruebas de infraestructura y API de ms-notification por una dependencia no disponible en el entorno de ejecución.

Tabla 2: Indicadores del proyecto

Indicador	Resultado
Historias de usuario implementadas	40 de 69 (58,0 %)
Historias planeadas para la versión 2	29 de 69
Historias Must del alcance actual	20 de 20 (4 reprogramadas a la versión 2)
Pruebas del backend (JUnit y xUnit)	460 aprobadas de 460
Pruebas de la aplicación (Jest)	138 aprobadas de 138
Pruebas del firmware (arnés y MQTT)	28 aprobadas de 28
Cobertura de líneas del dominio (.NET)	73,6 % a 97,6 %
Cobertura de líneas de la capa de aplicación (.NET)	90,5 % a 100 %
Cobertura de líneas de la aplicación (Jest)	73,9 %
Endpoints REST / tablas de la base de datos	65 / 31
Diagramas UML / entidad-relación	39 / 7

La cobertura de los servicios .NET se midió con coverlet (figura 4). Las capas de dominio y aplicación, donde están las reglas de negocio, superan el 73 % en todos los servicios y el 90 % en la mayoría; las capas de infraestructura y API solo tienen proyectos de prueba en dos servicios y su cobertura es menor, porque gran parte de ese código es configuración y arranque.

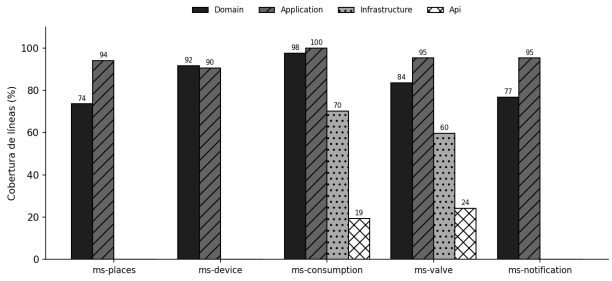


Figura 4: Cobertura de líneas por servicio y capa en los servicios .NET

6.3 Firmware

El arnés del firmware ejecutó seis escenarios: configuración inicial, WiFi incorrecta, operación normal, reinicio, red ausente y reloj sin sincronizar. Las 28 verificaciones pasaron, entre ellas que una orden duplicada se ejecuta una sola vez, que un medidor no recibe las órdenes de otro, que las lecturas guardadas sin red llegan en orden al reconectarse, que la válvula vuelve cerrada después de un corte de energía y que el botón de configuración funciona aunque la red guardada no esté disponible.

7 DISCUSIÓN

7.1 Impacto de la arquitectura hexagonal

La separación entre dominio y tecnología fue la decisión con mayor efecto en la calidad. Como los casos de uso solo conocen interfaces, reglas críticas como el bloqueo después de cinco intentos, el emparejamiento del medidor o el vencimiento de una orden de la válvula se probaron con dobles en memoria y un reloj controlado, en milisegundos y sin SQL Server, RabbitMQ ni hardware. Eso explica coberturas superiores al 90 % en la capa de aplicación con un equipo de tres personas.

7.2 Integración por eventos

La mensajería desacopló el medidor del resto del sistema: el servicio de dispositivos publica lo que ocurre sin conocer a quién le interesa, y agregar un nuevo consumidor, como el futuro detector de fugas, no requiere modificarlo. El costo es una mayor complejidad operativa (broker, colas de mensajes muertos y consumidores idempotentes), que en un proyecto formativo se compensó con convenciones comunes en los cinco servicios .NET.

7.3 Lecciones aprendidas

- Un defecto del firmware (el botón de configuración no respondía con el medidor alimentado por su fuente) no aparecía con el computador conectado; desde entonces el firmware se prueba siempre sin él, y el arnés incluye ese escenario.
- La documentación escrita antes de implementar se desactualiza: el servicio dueño de las tarifas y los nombres de algunos schemas cambiaron. La documentación se actualiza ahora en el mismo cambio que el código.
- Adaptar la aplicación móvil al escritorio con archivos específicos para la web evitó regresiones en el celular.

7.4 Limitaciones

El sistema no se ha validado todavía con usuarios reales ni se midieron tiempos de respuesta o disponibilidad, por lo que los resultados se limitan a la corrección funcional verificada por pruebas. Con un solo sensor a la entrada no es posible desagregar el consumo por punto de uso, y la detección de fugas planeada, basada en la duración del caudal, podrá confundir un uso prolongado legítimo, como el riego, con una fuga. Además, la copia revisada el 10 de octubre no incluía imágenes Docker de los servicios (se agregaron después del corte), MQTT con TLS ni la consulta de la lista de revocación en el servicio de dispositivos.

7.5 Relación con trabajos previos

A diferencia del sistema de Chang et al. [5], que usa dos medidores para detectar fugas por diferencia de caudal, Cuida Tu Agua usa un solo sensor y plantea compensar esa limitación con reglas de tiempo y horario en la versión 2; lo que ya agrega es el control de la válvula y la gestión de varios lugares por usuario. Frente a las soluciones revisadas por Velayudhan et al. [3], centradas en la red de distribución, el proyecto se enfoca en el usuario final, que es quien recibe el beneficio de la retroalimentación descrito por Schultz et al. [2].

8 CONCLUSIONES Y TRABAJO FUTURO

Se desarrolló una plataforma IoT web y móvil que permite conocer el consumo de agua en tiempo real, recibir alertas y cerrar la válvula de forma remota con confirmación de seguridad. La combinación de microservicios con arquitectura hexagonal, integración por eventos y un medidor de bajo costo resultó viable para un equipo de tres aprendices: al 15 de octubre, 40 historias implementadas, todas las historias Must del alcance actual y 626 pruebas automatizadas aprobadas.

Como trabajo futuro, en la versión 2, se plantea implementar la detección de fugas y el cierre automático con confirmación del dueño, y la comparación con el recibo del acueducto; asegurar el despliegue en contenedores con HTTPS y MQTT con TLS; medir el

rendimiento y la disponibilidad; y realizar un piloto con usuarios reales para evaluar su efecto en el consumo.

AGRADECIMIENTOS

Los autores agradecen al Servicio Nacional de Aprendizaje (SENA) y a los instructores del programa Tecnología en Análisis y Desarrollo de Software por su acompañamiento durante el proyecto.

REFERENCIAS

[1] Superintendencia de Servicios Públicos Domiciliarios. 2025. *Informe sectorial de los servicios públicos domiciliarios de acueducto y alcantarillado: vigencia 2024*. SSPD, Bogotá, Colombia.

[2] P. Wesley Schultz, Shahram Javey, and Alla Sorokina. 2019. Social comparison as a tool to promote residential water conservation. *Frontiers in Water* 1, Article 2. https://doi.org/10.3389/frwa.2019.00002

[3] Nibi Kulangara Velayudhan, Preeja Pradeep, Sethuraman N. Rao, Aryadevi Remanidevi Devidas, and Maneesha Vinodini Ramesh. 2022. IoT-enabled water distribution systems—A comparative technological review. *IEEE Access* 10 (2022), 101042–101070. https://doi.org/10.1109/ACCESS.2022.3208142

[4] Elias Farah and Isam Shahrour. 2024. Water leak detection: A comprehensive review of methods, challenges, and future directions. *Water* 16, 20 (2024), 2975. https://doi.org/10.3390/w16202975

[5] Chih-Chun Chang, Yao-Yu Lee, Ting-Yu Hou, and Chin-Chung Yu. 2022. Information security in wireless water flow and leakage alarm system. *Sensors and Materials* 34, 6 (2022), 2189–2197. https://doi.org/10.18494/SAM3811

[6] Nitin Naik. 2017. Choice of effective messaging protocols for IoT systems: MQTT, CoAP, AMQP and HTTP. In *2017 IEEE International Systems Engineering Symposium (ISSE)*. IEEE, 1–7. https://doi.org/10.1109/SysEng.2017.8088251

[7] Andrew Banks and Rahul Gupta (Eds.). 2014. *MQTT Version 3.1.1*. OASIS Standard. http://docs.oasis-open.org/mqtt/mqtt/v3.1.1/mqtt-v3.1.1.html

[8] Sam Newman. 2021. *Building Microservices: Designing Fine-Grained Systems* (2nd ed.). O'Reilly Media, Sebastopol, CA.

[9] Alistair Cockburn. 2005. Hexagonal architecture. Retrieved from https://alistair.cockburn.us/hexagonal-architecture/

[10] Ken Schwaber and Jeff Sutherland. 2020. *The Scrum Guide*. Retrieved from https://scrumguides.org/

[11] IEEE. 1998. *IEEE Recommended Practice for Software Requirements Specifications*. IEEE Std 830-1998. https://doi.org/10.1109/IEEESTD.1998.88286

[12] ISO/IEC. 2023. *Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — Product quality model*. ISO/IEC 25010:2023.

[13] Michael Jones, John Bradley, and Nat Sakimura. 2015. *JSON Web Token (JWT)*. RFC 7519. IETF. https://doi.org/10.17487/RFC7519

[14] Mark Nottingham, Erik Wilde, and Sanjay Dalal. 2023. *Problem Details for HTTP APIs*. RFC 9457. IETF. https://doi.org/10.17487/RFC9457